Cheatsheet AZ-104 — 10 · Contenedores: Container Registry, Container Instances y Container Apps

1. Elegir el servicio

Servicio	Qué es	Cuándo	No vale si
Azure Container Instances (ACI)	Contenedores **sueltos** (container groups) sin orquestador, por segundo	Tareas puntuales, batch, pruebas, **un contenedor rápido**, desbordamiento de AKS (virtual nodes)	Hace falta **autoscale**, balanceo entre réplicas, revisiones o ingress HTTP gestionado
Azure Container Apps (ACA)	Microservicios y apps HTTP / eventos **serverless** sobre Kubernetes gestionado (KEDA, Dapr, Envoy), **escala a cero**	APIs, webs en contenedor, workers por cola, jobs; sin administrar clúster	Necesitas acceso a la API de Kubernetes, Windows containers o control del nodo
AKS	Kubernetes completo	Control total, operadores, cargas complejas	Quieres algo simple (AZ-104 casi no lo pregunta)
App Service (Web App for Containers)	PaaS web con imagen propia	Web clásica en contenedor con slots, dominios, TLS	Multi-contenedor complejo, escala a cero
Azure Container Registry (ACR)	Registro **privado** de imágenes (Docker / OCI)	Siempre que uses lo anterior con imágenes propias	—

2. Azure Container Registry

Aspecto	Basic	Standard	Premium
Almacenamiento incluido / máximo	10 GiB / 40 TiB	100 GiB / 40 TiB	**500 GiB** / 100 TiB
Webhooks	2	10	500
Geo-replication (un registro, varias regiones)	—	—	**Sí**
Private Link (private endpoints), reglas de IP, service endpoints	—	—	**Sí**
Content trust (imágenes firmadas), **customer-managed keys**, **retention policy** (manifiestos sin tag), dedicated data endpoints, connected registries	—	—	**Sí**
Anonymous pull, cache de artefactos	—	Sí	Sí
Zone redundancy, tokens y scope maps, Entra RBAC por repositorio	Sí	Sí	Sí
Rendimiento (r/m, concurrencia)	Bajo	Medio	Alto

- Nombre: 5-50 alfanuméricos, **global**; login server `<nombre>.azurecr.io`. Cambiar de SKU no tiene downtime (bajar desde Premium exige quitar geo-réplicas).
- **Autenticación:** Entra ID con roles **AcrPull** (solo descargar), **AcrPush** (subir y descargar), **AcrDelete**, **AcrImageSigner**; **managed identity** para AKS / ACA / App Service / VMs; **service principal** para CI/CD; **admin user** (dos passwords; desactivado por defecto; solo pruebas); tokens con scope maps para repositorios concretos. "Least privilege for AKS to pull" → AcrPull a la identidad del clúster.
- **ACR Tasks:** `az acr build -r acr1 -t app:v1 .` construye en la nube **sin Docker local**; tareas automáticas por **commit** en Git, por **actualización de la imagen base** o programadas; multi-step tasks (YAML); `az acr run`. **Importar** de Docker Hub u otro registro: `az acr import -n acr1 --source docker.io/library/nginx:latest -t nginx:latest`.
- Operar: `az acr login -n acr1` → `docker tag app acr1.azurecr.io/app:v1` → `docker push acr1.azurecr.io/app:v1` · `az acr repository list -n acr1` · `az acr repository show-tags -n acr1 --repository app` · `az acr repository delete -n acr1 --image app:v1` · `az acr show-usage`. **Defender for Containers** escanea vulnerabilidades. **Image lock** (`az acr repository update --write-enabled false`) evita sobrescribir o borrar un tag.

3. Azure Container Instances

Aspecto	Detalle
Container group	Unidad de despliegue: uno o varios contenedores (hasta 60, **multi-contenedor solo Linux**) que comparten ciclo de vida, red (misma IP y puertos) y volúmenes; patrón sidecar; **init containers**
Recursos	Por contenedor se fijan **CPU y memoria** (`--cpu 1 --memory 1.5`); el grupo puede llegar a decenas de vCPU, pero por encima de 4 vCPU / 16 GB depende de capacidad regional; se paga **por segundo** de vCPU y GB
Imágenes	Linux (x64) y **Windows**; desde ACR con `--registry-login-server --registry-username --registry-password` o **managed identity** (`--acr-identity`)
Red	**IP pública** con etiqueta DNS (`--dns-name-label app1` → `app1.<region>.azurecontainer.io`) y puertos (`--ports 80 443`, máx. 5 por IP) **o** desplegado en una **VNet** (`--vnet --subnet`, subred delegada a `Microsoft.ContainerInstance/containerGroups`, IP **privada**; no ambas)
Almacenamiento	**Azure Files** (`--azure-file-volume-account-name/-key --azure-file-volume-share-name --azure-file-volume-mount-path`; solo Linux), emptyDir, gitRepo, secret volumes
Reinicio	`--restart-policy Always` (default; servicios) · `OnFailure` (tareas) · `Never` (ejecutar una vez)
Configuración	Variables `--environment-variables`, secretas `--secure-environment-variables`, comando `--command-line`; YAML (`az container create -f grupo.yaml`) para multi-contenedor
Operar	`az container logs -n g1 -g RG1` · `az container attach` (stdout en vivo) · `az container exec -n g1 --exec-command "/bin/sh"` · `az container show --query ipAddress.fqdn` · `az container restart / stop / start / delete`
Escalado	**No hay autoscale**: crear más grupos (o usar ACA / AKS); **Spot containers** (preview) y **confidential containers**

Aspecto	Detalle
Cuotas	Por defecto 100 container groups y 100 vCPU por región y suscripción (ampliables); sin réplicas ni redundancia integradas (HA = varios grupos tras tu propio balanceador)

- Caso típico: "run a one-off job / build / batch without managing VMs, billed per second" → ACI con `--restart-policy Never`. "Persistent data across restarts" → volumen de Azure Files. "Accessible only inside the VNet" → desplegar en la subred delegada.

4. Azure Container Apps

Pieza	Qué es	Detalle
Environment	Frontera segura compartida (red, Log Analytics, Dapr) para varias apps	**Consumption only** o **workload profiles** (Consumption + **Dedicated** D4/D8/E4... con más CPU y GPU); puede ir en tu **VNet** (interna o externa); apps del mismo environment se hablan por nombre
Container app	Imagen (Linux x64), recursos, secretos, identidad, **ingress**, escalado	Varios contenedores por app (sidecar / init) solo si están acoplados; `az containerapp up` crea environment + app en un paso
Revisions	Cada cambio del `template` (imagen, env, recursos) crea una **revisión** inmutable; modo **single** (la nueva sustituye) o **multiple** (varias activas)	**Traffic splitting** por % entre revisiones (blue-green, canary), etiquetas (labels) con URL propia; "roll back" = volver a enviar tráfico a la anterior
Ingress	**External** (público, HTTPS gestionado, dominio propio y certificado), **Internal** (solo environment / VNet) o deshabilitado; HTTP, TCP y gRPC; target port; restricciones de IP; sticky sessions	"only reachable from other apps" → internal
Scaling	**Min / max réplicas** (min 0 = **scale to zero**; máx. 1 000 réplicas); reglas **HTTP** (requests concurrentes), **TCP**, **custom** por scalers de **KEDA** (Service Bus, Storage queue, Event Hubs, CPU, memoria...). CPU / memoria **no escalan a cero**	Sin reglas: 1 réplica fija entre min y max; cooldown y polling configurables
Recursos	Combinaciones fijas en Consumption: **0,25 vCPU / 0,5 Gi** ... hasta **4 vCPU / 8 Gi** (memoria = 2 × CPU); workload profiles dedicados permiten más; en un environment *Consumption only* el máximo es 2 vCPU / 4 Gi	"0.5 CPU and 2 GiB" no es válido (sería 0,5 / 1 Gi)
Jobs	Contenedores que **terminan**: manual, **programado** (cron) o **disparado por evento** (KEDA)	Sustituyen a ACI para tareas dentro del environment
Secretos e identidad	Secretos propios o referencias a **Key Vault**; **managed identity** (system / user) para ACR (`acrPull`) y otros servicios	"no credentials in the app"
Dapr	Service invocation, pub/sub, state stores	Opcional
Logs	Consola y sistema en **Log Analytics** (`ContainerAppConsoleLogs_CL`, `ContainerAppSystemLogs_CL`) o Azure Monitor; `az containerapp logs show`	—

```
az containerapp env create -n env1 -g RG1 -l westeurope [--infrastructure-subnet-resource-id <subnet>] [--internal-only true]
az containerapp create -n app1 -g RG1 --environment env1 --image acr1.azurecr.io/app:v1 \
   --registry-server acr1.azurecr.io --registry-identity system --target-port 80 --ingress external \
   --min-replicas 0 --max-replicas 10 --cpu 0.5 --memory 1.0Gi --scale-rule-name http --scale-rule-http-concurrency 50
az containerapp update -n app1 -g RG1 --image acr1.azurecr.io/app:v2            # nueva revisión
az containerapp revision list -n app1 -g RG1 -o table
az containerapp ingress traffic set -n app1 -g RG1 --revision-weight app1--v1=80 app1--v2=20
az containerapp job create -n job1 -g RG1 --environment env1 --trigger-type Schedule \
   --cron-expression "0 2 * * *" --image ... --replica-timeout 1800
```

5. Comandos que caen (ACR y ACI)

Tarea	Azure CLI	PowerShell
Crear registro	`az acr create -n acr1 -g RG1 --sku Premium --admin-enabled false`	`New-AzContainerRegistry -ResourceGroupName RG1 -Name acr1 -Sku Premium`
Construir y subir	`az acr build -r acr1 -t app:v1 .` · `az acr login -n acr1` + `docker push acr1.azurecr.io/app:v1`	`Connect-AzContainerRegistry -Name acr1` + docker push
Importar y listar	`az acr import -n acr1 --source docker.io/library/nginx:latest -t nginx:latest` · `az acr repository list -n acr1` · `az acr repository show-tags -n acr1 --repository app`	`Get-AzContainerRegistryRepository -RegistryName acr1` · `Get-AzContainerRegistryTag -RegistryName acr1 -RepositoryName app`
Geo-replicar	`az acr replication create -r acr1 -l northeurope`	`New-AzContainerRegistryReplication -RegistryName acr1 -Location northeurope`
Permiso de pull	`az role assignment create --assignee <mi id> --role AcrPull --scope <acr id>`	`New-AzRoleAssignment -ObjectId <mi> -RoleDefinitionName AcrPull -Scope <acr id>`
Crear ACI pública	`az container create -g RG1 -n c1 --image acr1.azurecr.io/app:v1 --cpu 1 --memory 1.5 --dns-name-label app1 --ports 80 --os-type Linux --restart-policy Always --acr-identity <id> --assign-identity <id>`	`$p = New-AzContainerInstancePortObject -Port 80 -Protocol TCP; $c = New-AzContainerInstanceObject -Name c1 -Image ... -RequestCpu 1 -RequestMemoryInGb 1.5 -Port $p; New-AzContainerGroup -ResourceGroupName RG1 -Name c1 -Location westeurope -Container $c -OsType Linux -IPAddressType Public -IPAddressDnsNameLabel app1 -RestartPolicy Always`
ACI en VNet	`az container create ... --vnet vnet1 --subnet sub-aci` (sin `--dns-name-label`)	`-SubnetId <id>`
ACI con Azure Files	`az container create ... --azure-file-volume-account-name st1 --azure-file-volume-account-key <key> --azure-file-volume-share-name share1 --azure-file-volume-mount-path /data`	`New-AzContainerGroupVolumeObject -Name v1 -AzureFileShareName share1 -AzureFileStorageAccountName st1 -AzureFileStorageAccountKey <key>`

Tarea	Azure CLI	PowerShell
Operar ACI	`az container logs / attach / exec / show / restart / delete -g RG1 -n c1`	`Get-AzContainerInstanceLog` · `Get-AzContainerGroup` · `Restart-AzContainerGroup` · `Remove-AzContainerGroup`

6. Árbol de decisión

```
¿Qué pide el enunciado?
├─ Guardar imágenes propias de forma privada ........................... Azure Container Registry
├─ Registro en varias regiones / private endpoint / imágenes firmadas ... ACR Premium
├─ Construir la imagen sin Docker en el equipo ......................... ACR Tasks (az acr build)
├─ Que AKS / ACA / App Service tiren imágenes sin credenciales ......... Managed identity + AcrPull
├─ Un contenedor rápido, tarea o batch, pago por segundo ............... ACI (restart-policy Never / OnFailure)
├─ Contenedor con IP privada en la VNet ................................ ACI en subred delegada (o ACA con VNet)
├─ Varios contenedores que comparten ciclo de vida y localhost ......... ACI container group (Linux) o ACA sidecar
├─ Persistir datos de un contenedor .................................... Volumen Azure Files
├─ API HTTP que escala según carga y hasta cero ........................ Container Apps (ingress + reglas HTTP)
├─ Procesar mensajes de una cola, escalar por longitud ................. Container Apps con scaler KEDA
├─ Desplegar v2 al 10 % de usuarios y volver atrás ..................... Container Apps: revisiones múltiples + traffic split
├─ Tarea programada en contenedor ...................................... Container Apps Job (schedule)
├─ Solo accesible desde otras apps del environment ..................... Ingress internal
└─ Kubernetes completo con acceso a kubectl ............................ AKS
```

7. Señales → respuesta

Enunciado	Respuesta
"private registry with geo-replication"	ACR Premium
"registry must not be reachable from the internet"	ACR Premium + private endpoint (y public access disabled)
"least privilege for the container app to pull images"	AcrPull a su managed identity
"admin user credentials in the deployment"	Mala práctica: usar identidad / service principal
"build the image in Azure from the Dockerfile"	`az acr build`
"automatically rebuild when the base image updates"	ACR Task con trigger de base image
"delete untagged manifests automatically"	Retention policy (Premium)
"run a container for a few minutes and bill only for that"	ACI, restart policy Never
"container needs a public DNS name"	ACI con `--dns-name-label`
"container group with Windows and Linux containers"	Imposible: multi-contenedor solo Linux
"ACI must access resources in the VNet only"	ACI en subred delegada (sin IP pública)
"data must survive container restarts"	Azure Files volume
"scale to zero when idle"	Container Apps (min replicas 0)
"scale on HTTP concurrent requests / queue length"	Container Apps scale rule HTTP / custom (KEDA)
"0.5 vCPU with 2 GiB" rejected	Combinaciones fijas: 0,5 vCPU → 1 Gi
"run two versions and split traffic"	Multiple revision mode + traffic weights
"container app only reachable inside the environment"	Ingress internal
"nightly job in a container inside the environment"	Container Apps Job scheduled
"ACI needs more replicas automatically"	No autoscale en ACI → ACA

8. Errores a evitar

1. Elegir ACI cuando piden autoscale, revisiones o escala a cero: eso es Container Apps.
2. Esperar Private Link, geo-replicación o content trust en ACR Basic / Standard.
3. Usar el admin user de ACR en producción en lugar de managed identity + AcrPull.
4. Pedir IP pública y VNet a la vez en un container group, o multi-contenedor en Windows.
5. Olvidar el volumen de Azure Files y perder datos al reiniciar.
6. Configurar CPU / memoria fuera de las combinaciones permitidas en Container Apps (memoria = 2 × vCPU).
7. Pensar que CPU o memoria permiten escalar a cero en Container Apps (solo HTTP, TCP y scalers de eventos).
8. Dejar ingress external cuando el enunciado pide acceso solo interno.